

Reducing inference cost in SeisBench through reduced precision and compilation

Constantinos Skevofilax

April 2026

1 Background

1.1 How SeisBench’s `annotate()` function works

SeisBench’s `annotate()` is a high-throughput function that performs inference on multiple waveforms given a pretrained model. `annotate()` can apply standard preprocessing to the waveforms, including filtering, resampling, normalization, and any optional augmentations the model needs in order to match the assumptions of its training set, particularly the sampling rate and the amplitude scaling.

Post-preprocessing, each trace is broken into a sequence of windows, where a window is a contiguous chunk of samples whose length matches the model’s receptive field. PhaseNet and PhaseNetLight expect a window of at least 3001 samples, while EQTransformer and EQT-NC expect at least 6000 samples. Once the windows are ready they are stacked into batches and fed through the neural network in one forward pass per batch, so that the matrix multiplications can amortize the fixed cost of dispatch instead of paying for it on every window separately.

Preprocessing and inference can both run on the CPU, but if a GPU is available, SeisBench will perform preprocessing on the CPUs and run the forward pass on the GPU. We prefer to run the inference on a GPU rather than on the CPU because of the GPU’s designed purpose for parallel processing. A modern CPU has on the order of 4 to 24 cores, with 8 being a typical figure for general-purpose machines. On the other hand, a single NVIDIA RTX 6000 Ada Generation GPU (which is the GPU that we have at TexNet) has 18,176 CUDA cores. With that many more cores, we can keep as many threads as busy as possible while pushing large batches of windows through convolutional and attention layers in roughly the same time it would take a CPU to process one batch of windows.

Probabilities come out of the model on a per-batch basis and SeisBench then rearranges them into the per-sample picks that users can see as a related pick. `annotate()` relies on Python’s `asyncio` library to manage preprocessing and model inferences through cooperative multitasking. Multiple tasks can run

concurrently within a single thread. An entire program doesn't have to wait for a single inference to complete, rather the event loop will switch to other ready tasks, resuming the original one only once its external dependency is satisfied. This is how SeisBench uses `annotate()` to perform quick batch inference on multiple waveforms.

Additionally, every arithmetic operation inside that forward pass uses 32-bit floating point precision (FP32). This is important as to where we can save compute time, as FP32 is quite expensive computationally to maintain that level of precision per computation.

1.2 Where speed gains are still on the table

Even though `annotate()` is already efficient (Table 1), there is room to go even faster. The cleanest method is change the numerical precision used during inference itself. PyTorch on a modern GPU has available two reduced-precision formats: FP16 (half precision from FP32) and BF16 (bfloat16). Both store fewer mantissa bits than FP32, where the mantissa is the part of the floating-point representation that holds the significant digits of the number. Using fewer mantissa bits cuts the memory traffic per activation and per weight read in half using either FP16 or BF16, which is what enables the speedup on hardware. The two formats have different compute time saving tradeoffs. FP16 keeps a 5-bit exponent, which is enough range for most well-scaled tensors but narrow enough that it can underflow or overflow on activations with large dynamic range. BF16 keeps the same 8-bit exponent that FP32 uses and pays the price entirely in mantissa width, so it preserves the dynamic range that some models rely on. Inside both formats, only the convolutional and linear layers are cast to the reduced precision under an autocast policy, while sensitive reductions such as softmax and normalization stay in FP32 for stability.

Additionally, there is a second option for improving speedup, which is graph compilation. PyTorch's `torch.compile` can be applied on top of either FP32 or BF16 to fuse adjacent operations into single kernels, which removes the sequential processing the Python does - it reads every line of code sequentially. By compiling, those mathematical operators that make up the neural network are stacked into a smaller number of larger operation kernels that the GPU can run autonomously without having to read the subsequent line of code. Without this, the GPU has to sit idly for a few microseconds on each operator while Python computes the next one.

By combining these two techniques, we can get faster processing speeds than what SeisBench has available.

2 Methods

2.1 Inference precision and compilation axes

To check whether we achieve faster processing than `annotate()`, we benchmarked `annotate()` along with various PyTorch implementations that used different numerical precision and compilation techniques previously mentioned: FP16 and BF16.

FP16 was not utilized on any EQT models. This is because using FP16 on either model produced numerical incompatibility errors due to some architectural padding values inherent in the models utilize a higher level of precision than what FP16 can offer. While we couldn't perform FP16 precision inference with any EQT models, we could use BF16, as it kept the FP32 exponent range, sidestepping the overflow problem with FP16 while still tightening the mantissa enough to gain speedup. The PhaseNet family has no such issue and was tested in all three precisions.

On top of using either FP32 or BF16, we also enabled `torch.compile` wherever it was stable. In practice, this means only on single-GPU execution, as the compiled graphs are not stable when using threaded multi-GPU processing.

2.2 Hardware placement and execution mode

Unless explicitly noted, preprocessing executed solely on the CPU and the forward passes executed on the GPU. The dual-GPU trials split each megabatch across both GPUs evenly using in-process threads rather than separate processes, which keeps the model weights resident on both devices for the duration of the trial and avoids paying the cost of moving them in and out of device memory between calls.

2.3 Timing and pick-quality metrics

For every trial we logged a wall-clock measurement of the inference segment, and where it makes sense we also log stage-level timers that separate preprocessing from the forward pass. Pick quality is reported alongside timing so that we can verify that none of the precision changes silently degrade what the model produces. We compare the predicted onset index against the catalog pick that was duplicated across the synthetic stations and report the absolute offset in samples at the 100 Hz sampling rate.

2.4 Datasets, models, and sweep geometry

As our input data, we used traces from the STEAD and TXED datasets. We ran each trial on the pretrained PhaseNet, PhaseNetLight, EQTransformer, and EQTransformer Non-Conservative (EQT-NC) models existing on SeisBench. To test runtimes against various station counts, we duplicated the 3-C traces for a given event to equal the amount of 3-C waveforms needed to be equal to a given

number of stations. We tested varied station counts, and sub-batch sizes. The station counts swept were 64, 256, and 580, and at each station count we swept sub-batch sizes of 64, 256, 512, and 1024. Each trial was repeated twice with the same random seed so that the same waveforms that went through inference both times gave us enough variance for a given data point to reliably report it. For each dataset and station-count combination, we evaluated it against 100 different event traces, which is enough to observe how reduced precision shifts the pick quality distribution without inflating the runtime of the sweep itself.

3 Results

No errors occurred during our trials. As mentioned earlier, we skipped all trials involving the EQT models with FP16, as well as dual-GPU trials that requested `torch.compile`, which we omit because threaded multi-GPU execution with Inductor and CUDA graphs is unstable. Across the four artifacts we aggregate, the matrix produced 333,655 timed rows, with all skips clearly attributed by the driver and none of them caused by runtime failures. Throughout this section, wall time refers to the recorded `wall_time_s` field of each row, peak GPU memory is taken from the per-device PyTorch allocator highwater (we report the higher of the two devices for the dual case), and peak CPU memory is reported as host RSS at the end of the trial. Pick quality is summarized as the median absolute onset offset, in samples at 100 Hz, between the predicted onset and the catalog pick that we replicated across the duplicated stations.

3.1 Baseline `annotate()` performance against various station counts

Median wall time grew as the station count increases for both CPU and single-GPU configurations, as expected. For increasing station count on both CPU and 1 GPU processing, the runtime curve grows in a near linear fashion, which is consistent with how `annotate()` treats each individual station: as an independent megabatch whose internal window count scales with N_{st} .

Surprisingly, the CPU ran the `annotate()` function faster than a single GPU for every cell of Table 1 except for one: PhaseNet at $N_{\text{st}} = 64$, where the GPU completed the processing in 0.072 s against the CPU’s 0.093 s. For PhaseNet-Light at $N_{\text{st}} = 64$ the CPU finishes at 0.037 s versus 0.047 s on one GPU, and for EQT the CPU finishes at 0.063 s versus 0.109 s on one GPU. The pattern holds at the larger station counts as well, where CPU at $N_{\text{st}} = 580$ is 0.306 s for PhaseNet against 0.335 s on one GPU, 0.292 s for PhaseNetLight against 0.321 s, 0.317 s for EQTransformer against 0.385 s, and 0.309 s for EQT-NC against 0.365 s. The most likely reason is the host configuration: our two AMD Threadripper PRO 7985WX CPUs supply 128 physical cores, so the BLAS-backed convolutions inside `annotate()` have plenty of parallel hardware available, while the single-GPU path still pays the per-window dispatch overhead that SeisBench’s window loop incurs in Python.

The threaded dual-GPU baseline is consistently slower than the single GPU at every station count we tested. This is most likely due to the threaded splitter we use to halve the station list evenly across `cuda:0` and `cuda:1` adds a coordination cost that scales with the work being split rather than being overpassed as work count grows. Even so, the main reason the dual baseline never achieves faster processing than 1 GPU is because `annotate()` already keeps a single GPU adequately fed on the megabatch of information, so the second device only gets half of an already-sufficient workload while paying for an extra host-to-device transfer.

The peak GPU memory of the baseline `annotate()` runs is small in every case, ranging from 7 to 11 MiB for PhaseNetLight, around 1.1 GiB for PhaseNet, and around 2.1 GiB for the EQTransformer family, with no growth across station counts because `annotate()` reuses the same internal buffer per call. The host RSS sits between 3.3 and 3.7 GiB regardless of the model because SeisBench keeps the dataset cache and the duplicated station Stream in memory.

3.2 Single-GPU performance and reduced precision throughput

Once we bypass `annotate()` and feed minibatches directly to the model with the lean PyTorch path, the picture splits along model architecture. The PhaseNet variants get measurably faster on the GPU. At $N_{\text{st}} = 64$ the median wall time for PhaseNet on a single GPU is 0.017 s in BF16 and 0.016 s in FP16, and PhaseNetLight is 0.016 s in either precision, which is roughly $4\times$ shorter than the baseline 1-GPU row for PhaseNet (0.072 s) and roughly $3\times$ shorter for PhaseNetLight (0.047 s). At $N_{\text{st}} = 580$ the lean PhaseNet median is 0.147 s in BF16 against a 1-GPU baseline of 0.335 s, so the lean path is about $2.3\times$ faster than the baseline at the largest station count we tested for that model. The reduced precision modes do not move the median by much for the small PhaseNet variants because those models are already small enough that wall time is dominated by Python and CUDA launch overhead, not by the matmul cost itself.

The EQTransformer family goes the other direction. At $N_{\text{st}} = 64$ the lean BF16 median is 0.161 s for EQTransformer and 0.136 s for EQT-NC, both of which are above the 1-GPU baseline at the same tier (0.109 s and 0.089 s). At $N_{\text{st}} = 580$ the gap widens, with EQTransformer lean BF16 at 1.175 s versus 0.385 s for the baseline, and EQT-NC lean BF16 at 1.118 s versus 0.365 s for the baseline. We see the same pattern across batch sizes, with the largest sub-batch of 1024 producing the lowest lean median for the EQT family but still landing well above the baseline. Our reading is that SeisBench’s `annotate()` relies on internal kernel paths that the lean PyTorch loop does not reproduce on its own, especially for the attention-heavy EQTransformer layers, so the lean path loses for these models even though it removes the outer Python overhead. BF16 also stays close to FP32 in the compiled rows of Table 3 (within 1 ms on every model and station tier), which is consistent with the EQT family being limited by the fixed cost of the convolutional and attention layers rather than by memory bandwidth where BF16 usually wins. We could not run FP16 on the

EQTransformer family because of the numerical instabilities discussed earlier, so the EQT rows in Table 2 are BF16 only.

The peak GPU memory of the lean path is much lower than the baseline because we do not hold the full duplicated Stream on the device. With compilation off (Table 2), PhaseNet sits at 62 to 228 MiB across the three station counts in either FP16 or BF16 because the autocast scratch space dominates, PhaseNetLight is between 40 and 160 MiB, and the EQT family in BF16 climbs from 97 MiB at $N_{\text{st}} = 64$ to 441 MiB at $N_{\text{st}} = 580$. Pick quality is unchanged across precisions for every model, with PhaseNet recording a 13-sample median absolute onset offset and EQTransformer at 13 to 15 samples, which corresponds to roughly 130 to 150 ms at the 100 Hz sampling rate. PhaseNetLight has a wider 32 to 33-sample offset across all precisions, but the consistency of that number across FP16 and BF16 tells us reduced precision is not the source of the offset.

3.3 Impact of compilation and numerical precision on single-GPU CUDA

We restricted the `torch.compile` sweep to FP32 and BF16 forwards on a single GPU, and skipped FP16 because the autocast scratch space that FP16 requires inside the lean loop interacts poorly with Inductor’s tracing for these models on the PyTorch version we used. The dual-GPU runners were also excluded from the compile sweep for the threading-stability reasons noted earlier. As a result, every row in Table 2 corresponds to the lean path with compilation disabled, and the compile-enabled medians for FP32 and BF16 on a single GPU live in Table 3.

For the PhaseNet variants, compilation barely moves the median because the wall time is already dominated by Python and CUDA dispatch overhead rather than by the matmul itself. Without compilation, PhaseNet in BF16 sits at 0.147 s on a single GPU at $N_{\text{st}} = 580$ (Table 2); with compilation it lands at 0.146 s (Table 3), and at the smaller tiers the difference is comparable. PhaseNetLight in BF16 moves from 0.152 s without compile to 0.147 s with it at the same $N_{\text{st}} = 580$ tier, which is the largest single shift we see for the light models but still small in absolute terms. The compiled FP32 versions in Table 3 land within a millisecond of compiled BF16 on every PhaseNet row, which is well inside the noise we see across repeats. What this says is that for these light models the precision change in Table 2 is doing essentially all of the work, and the lean path beats `annotate()` comfortably on a single GPU regardless of whether we compile. Pooled across N_{st} , compiled BF16 PhaseNet is 0.064 s versus the 0.171 s `annotate` baseline (63% faster), and compiled BF16 PhaseNetLight is 0.063 s versus the 0.146 s baseline (57% faster).

The EQTransformer family is where compilation does measurable work. On a single GPU, EQTransformer BF16 drops from 0.519 s without compile (Table 2, pooled across N_{st}) to 0.481 s with it (Table 3, pooled), a 7.3% improvement on the median, and EQT-NC BF16 drops from 0.490 to 0.471 s, a 3.9% improvement. The FP32 versions improve by similar relative amounts: EQTransformer FP32 lands at 0.485 s once compiled, a 5.5% drop from the 0.513 s

pooled FP32 median without compile, and EQT-NC FP32 lands at 0.474s, a 2.7% drop. The catch is that even with compilation, the EQT family does not match `annotate()`. Compiled BF16 EQTransformer is 127% slower than the 0.212s annotate baseline, and compiled BF16 EQT-NC is 144% slower than the 0.193s annotate baseline. Compilation narrows the deficit (without it the same rows would be 145% and 154% slower) but does not close it. The pattern lines up with our reasoning that the small PhaseNet variants are limited by host overhead, so removing kernel-launch slack with Inductor does not gain anything, while the EQTransformer family has enough fused matmul work for the compiled graph to be measurably better but still not enough to beat what `annotate()` already does on the same hardware.

A separate observation worth flagging is that BF16 with compilation runs at essentially the same wall time as FP32 with compilation on every model in Table 3, so for these models the precision lever is independent of the compile lever and does not stack the way it would on a memory-bandwidth-limited workload. Where compilation does pay off cleanly is in peak GPU memory: at $N_{st} = 580$, PhaseNet drops from 228 MiB without compile to 60 MiB with it, and the EQTransformer family drops from roughly 440 MiB to 150 to 167 MiB, because the compiled graph is more efficient with intermediate activation buffers.

3.4 Threaded dual-GPU performance and current coverage

The threaded dual-GPU runners come in two variants: a serial driver that sends one shard to `cuda:0`, waits for it, and then sends the other shard to `cuda:1`, and a pipelined driver that fires the two shards as concurrent threads with a configurable CPU affinity per worker. The original sweep was missing every $N_{st} > 64$ row because of a resume-key collision in the matrix driver, which we fixed before re-running, so the data discussed here pools the original artifact with the continuation file that filled in $N_{st} \in \{256, 580\}$.

At $N_{st} = 64$ in BF16, the dual pipelined runner posts a median of 0.022s for PhaseNet and 0.026s for PhaseNetLight, with the dual serial runner landing within a few milliseconds of those values. The pipelined variant’s CPU affinity sweep across 12, 16, and 20 cores per GPU did not move the median by more than a few milliseconds on any model. Compared to SeisBench’s `annotate()` on a single GPU at the same $N_{st} = 64$, those dual lean BF16 medians are about 69% faster than baseline for PhaseNet (0.022s versus 0.072s) and 44% faster for PhaseNetLight (0.026s versus 0.047s), so for these two models the threaded dual configuration does beat `annotate()`. The EQT family goes the opposite direction. EQTransformer BF16 lands at 0.230s on two GPUs against `annotate()`’s 0.109s on one, a 112% increase in runtime, and EQT-NC lands at 0.184s against 0.089s, a 107% increase. The result is consistent with the EQT family results we saw in the single-GPU section: the lean code path does not match what `annotate()` achieves on the EQT models, and adding a second GPU does not change that conclusion.

The same separation holds at the larger station counts. PhaseNet BF16 on

two GPUs is 0.084 s at $N_{\text{st}} = 256$ and 0.209 s at $N_{\text{st}} = 580$, which is 50% and 38% faster than the corresponding `annotate()` 1-GPU baselines (0.171 s and 0.335 s). PhaseNetLight on two GPUs is 31% faster than `annotate()` at both $N_{\text{st}} = 256$ and $N_{\text{st}} = 580$. EQTransformer BF16 on two GPUs is 0.686 s at $N_{\text{st}} = 256$ and 1.443 s at $N_{\text{st}} = 580$, which is 224% and 275% slower than `annotate()`'s 0.212 s and 0.385 s baselines. EQT-NC follows the same pattern with 232% and 283% increases at the two larger tiers. The dual peak GPU memory rises sharply with N_{st} on the lean path, reaching 13.2 GiB for PhaseNet at $N_{\text{st}} = 580$, which reflects the extra activation buffers that the pipelined runner keeps live on each device. Reduced precision behaves the same way on the dual path as on a single GPU, with FP16 and BF16 essentially tied for the PhaseNet family. The takeaway is that a threaded two-GPU dispatch beats `annotate()` on the PhaseNet variants but loses to it on the EQT family, even at the largest station count we tested. A heavier model, or a process-based dispatcher that avoids the Python GIL during thread coordination, would be the natural next steps for revisiting this conclusion on the EQT family.

3.5 CPU lean throughput across precisions

The CPU lean path is included for completeness and as a reference point for users who do not have a GPU. Compared to SeisBench's `annotate()` on the same CPU (Table 1), the lean BF16 path is faster on the PhaseNet variants and considerably slower on the EQT family. PhaseNet in BF16 runs at 0.025 s at $N_{\text{st}} = 64$ against an `annotate()` median of 0.093 s (74% faster), 0.085 s at $N_{\text{st}} = 256$ against 0.165 s (48% faster), and 0.201 s at $N_{\text{st}} = 580$ against 0.306 s (34% faster). PhaseNetLight in BF16 is between 21% and 34% faster than `annotate()` at the three station counts. The EQTransformer family loses by a wide margin: EQTransformer BF16 is 196% slower than `annotate()` at $N_{\text{st}} = 64$, 321% slower at $N_{\text{st}} = 256$, and 370% slower at $N_{\text{st}} = 580$. EQT-NC follows the same pattern with 208%, 324%, and 363% increases at the three tiers.

FP16 on this CPU is a more uneven story for the PhaseNet variants because the host has AVX-512 BF16 instructions but no comparable native FP16 path, so FP16 still beats `annotate()` on PhaseNet at $N_{\text{st}} = 64$ (39% faster) but is 7% slower at $N_{\text{st}} = 256$ and 32% slower at $N_{\text{st}} = 580$, while PhaseNetLight in FP16 stays between 30% and 35% faster than `annotate()` at every tier. The EQT family was excluded from the CPU FP16 trials for the same reason it was excluded from the GPU FP16 trials. CPU peak RSS spans roughly 2.4 to 4.6 GiB across the lean rows, with a per-cell median near 3.6 GiB, because we still hold the duplicated Stream and the SeisBench cache in memory and the working set varies with N_{st} and the sub-batch size, even though the matmul activations themselves are small. Pick quality is the same as on the GPU runs, which gives us reassurance that none of our preprocessing or batching choices on the CPU path change the model output.

3.6 Limitations

EQTransformer-family FP16 trials are intentionally skipped on every device because the precision errors documented earlier in the paper would invalidate the resulting picks. Dual-GPU rows that requested `torch.compile` are also intentionally skipped because the threaded driver and the Inductor and CUDA graph backends are not stable together in the version of PyTorch we used. Pick-quality pairing assumes consistent catalog trace enumeration across phases, which is true by construction for the duplicated-station inputs we used here.

Table 1. Baseline SeisBench `annotate()` median wall time and peak memory per model and station count.

Model	N_{st}	CPU		1 GPU (cuda:0)		2 GPU (threaded)	
		wall (s)	mem (MiB)	wall (s)	mem (MiB)	wall (s)	mem (MiB)
PhaseNet	64	0.093	3,303	0.072	1,136	0.082	1,135
PhaseNet	256	0.165	3,495	0.171	1,136	0.213	1,135
PhaseNet	580	0.306	3,567	0.335	1,136	0.445	1,135
PhaseNetLight	64	0.037	3,620	0.047	7	0.057	11
PhaseNetLight	256	0.132	3,643	0.146	7	0.187	11
PhaseNetLight	580	0.292	3,695	0.321	7	0.417	11
EQTransformer	64	0.063	3,689	0.109	2,122	0.168	2,107
EQTransformer	256	0.158	3,666	0.212	2,122	0.291	2,107
EQTransformer	580	0.317	3,739	0.385	2,123	0.522	2,107
EQT-NC	64	0.053	3,719	0.089	2,122	0.115	2,107
EQT-NC	256	0.149	3,700	0.193	2,123	0.243	2,107
EQT-NC	580	0.309	3,735	0.365	2,123	0.471	2,107

Table 2. Lean PyTorch median wall time, peak memory, and pick quality per model, numerical precision, and station count, with `torch.compile` disabled. EQTransformer-family FP16 rows are intentionally omitted because the architectural padding values inside those models lie outside the finite range of FP16.

Model	Dtype	N_{st}	CPU		1 GPU		2 GPU (threaded)		Pick quality	
			wall (s)	mem (MiB)	wall (s)	mem (MiB)	wall (s)	mem (MiB)	$ \Delta_p $	$ \Delta_s $
PhaseNet	FP16	64	0.056	3,343	0.016	62	0.022	546	13	13
PhaseNet	FP16	256	0.177	3,566	0.065	227	0.088	463	13	13
PhaseNet	FP16	580	0.404	3,729	0.147	228	0.224	1,517	13	13
PhaseNet	BF16	64	0.025	3,302	0.017	62	0.022	120	13	13
PhaseNet	BF16	256	0.085	3,517	0.065	227	0.084	454	13	13
PhaseNet	BF16	580	0.201	3,567	0.147	228	0.209	684	13	13
PhaseNetLight	FP16	64	0.024	3,619	0.016	40	0.026	123	32	33
PhaseNetLight	FP16	256	0.086	3,641	0.063	159	0.101	475	32	33
PhaseNetLight	FP16	580	0.206	3,702	0.151	160	0.224	717	32	33
PhaseNetLight	BF16	64	0.025	3,619	0.016	40	0.026	123	32	33
PhaseNetLight	BF16	256	0.091	3,677	0.063	159	0.101	475	32	33
PhaseNetLight	BF16	580	0.231	3,750	0.152	160	0.222	717	32	33
EQTransformer	BF16	64	0.185	3,688	0.161	97	0.230	138	13	14
EQTransformer	BF16	256	0.664	3,708	0.519	306	0.686	523	13	14
EQTransformer	BF16	580	1.489	3,916	1.175	441	1.443	1,103	13	15
EQT-NC	BF16	64	0.165	3,699	0.136	97	0.184	138	14	15
EQT-NC	BF16	256	0.631	3,757	0.490	306	0.639	523	14	15
EQT-NC	BF16	580	1.431	3,916	1.118	440	1.397	1,035	13	15

Table 3. Lean PyTorch median wall time, peak memory, and pick quality with `torch.compile` enabled, on a single GPU only. The compile sweep was run for FP32 and BF16 forwards; FP16 was omitted because its autocast scratch space is unstable under Inductor for these models, and the dual-GPU runners were excluded because threaded multi-GPU dispatch is unstable with CUDA graphs.

Model	Dtype	N_{st}	wall (s)	mem (MiB)	$ \Delta_p $	$ \Delta_s $
PhaseNet	FP32	64	0.016	20	13	13
PhaseNet	FP32	256	0.064	60	13	13
PhaseNet	FP32	580	0.146	60	13	13
PhaseNet	BF16	64	0.016	20	13	13
PhaseNet	BF16	256	0.064	60	13	13
PhaseNet	BF16	580	0.146	60	13	13
PhaseNetLight	FP32	64	0.016	14	32	33
PhaseNetLight	FP32	256	0.063	54	32	33
PhaseNetLight	FP32	580	0.148	54	32	33
PhaseNetLight	BF16	64	0.016	14	32	33
PhaseNetLight	BF16	256	0.063	54	32	33
PhaseNetLight	BF16	580	0.147	54	32	33
EQTransformer	FP32	64	0.123	100	13	14
EQTransformer	FP32	256	0.485	110	13	14
EQTransformer	FP32	580	1.091	146	13	14
EQTransformer	BF16	64	0.123	86	13	14
EQTransformer	BF16	256	0.481	96	13	15
EQTransformer	BF16	580	1.083	166	13	15
EQT-NC	FP32	64	0.121	51	13	15
EQT-NC	FP32	256	0.474	100	13	15
EQT-NC	FP32	580	1.081	167	13	15
EQT-NC	BF16	64	0.120	46	13	15
EQT-NC	BF16	256	0.471	89	13	15
EQT-NC	BF16	580	1.075	150	13	15

Table 4. Per-artifact row counts, timed rows, and documented skips.

Artifact	Total rows	Timed rows	Errors	Skip reasons (count)
seisbench_matrix_lean_1gpu.jsonl	100,800	91,200	0	eqt_lean_fp16: 9,600
seisbench_matrix_lean_dual_gpu_threading.jsonl	129,600	65,600	0	dual_gpu_torch_compile_threads: 51,200; eqt_lean_fp16: 12,800
seisbench_matrix_lean_dual_gpu_threading_cont.jsonl	241,798	124,110	0	dual_gpu_torch_compile_threads: 95,524; eqt_lean_fp16: 22,110
seisbench_matrix_lean_cpu.jsonl	62,400	52,800	0	eqt_lean_fp16: 9,600